Final Project Report: Autonomous AI Job Orchestrator

Authors: J. A. L. Perera & A. A. Lokuliyana
Project: Final Year Project - Autonomous AI Job Orchestrator
Date: April 2026

---

1. Executive Summary

For our final project, we built the **Autonomous AI Job Orchestrator**. In simple terms, this is a highly intelligent "manager" for computer tasks. Whenever a computer or a large server farm has hundreds of different tasks to complete (like saving files, processing video, or updating databases), it needs a scheduler to decide *what* to do and *when* to do it.

We replaced traditional, rigid scheduling rules with a **Deep Reinforcement Learning Artificial Intelligence (AI)**. By doing this, we created a system that actively learns how to organize workloads mathematically, ensuring that high-priority tasks are completed quickly without completely ignoring or "starving" low-priority tasks.

---

2. The Core Problem: Why Old Schedulers Fail

Before we explain our solution, it is important to understand how traditional servers handle tasks and why they fail under heavy stress. Normally, servers use simple rules:

1. **FIFO (First-In-First-Out):** This is like a grocery store checkout line. Whoever arrived first gets served first. The problem? If a life-or-death priority task arrives, it has to wait in line behind 50 useless background updates just because they arrived earlier. It is completely blind to urgency.
2. **Strict Priority:** This fixes the FIFO problem by creating a "VIP Line". High-priority tasks instantly jump to the front. But this creates a catastrophic problem called **Starvation**. If high-priority tasks keep arriving, the low-priority tasks get pushed back forever. Eventually, their deadlines pass, and the system fails them.

Our Goal: We wanted to build a scheduler that doesn't just blindly follow rules. We wanted a scheduler that acts like a human manager—looking at the whole picture, doing priority tasks first, but squeezing in low-priority tasks when there is a gap so that nobody gets left behind.

---

3. How Our System Works (The 4 Pillars)

To make this work in a real enterprise environment, we did not just build a math algorithm; we built an entire distributed software architecture. It relies on four main pieces of technology collaborating in real-time.

Pillar 1: The Front Desk (FastAPI)

We built an API gateway using FastAPI. Think of this as the receptionist. Any user, computer, or external software that wants a job done submits it to this API. The API checks if the job makes sense (Does it have a duration? Does it have a valid priority number?) and assigns it a unique digital ID.

Pillar 2: The Master Whiteboard (Redis Database)

Once the API accepts a job, it writes it onto a digital whiteboard called Redis. Redis is an ultra-fast memory database. It holds the "State" of every job (Pending, Running, Completed, or Failed). Because Redis is a separate database, if our main program suddenly crashes and restarts, it can look at Redis and pick up exactly where it left off without losing any jobs.

Pillar 3: The Employees (Distributed Workers)

The workers are the background scripts that actually perform the physical computer tasks. We decoupled them from the main brain using Docker. This means if we have too much work, we can instantly clone our workers (e.g., scale up from 2 workers to 10 workers) without changing a single line of code. They constantly look at Redis and say, "Give me the next task the AI wants me to do."

Pillar 4: The Brain (AI Scheduler / Deep Q-Network)

This is the core of our project. Sitting between the Pending jobs and the Workers is our AI Brain. It constantly scans the queue. Instead of just looking at priority, it looks at:

- **Priority:** How important is this?
- **Duration:** How long will this take to finish?
- **Slack Time:** How many seconds do we have left before we miss the absolute deadline?

---

4. The Lifecycle of a Job (What Actually Happens Inside)

When you turn on the program and submit 50 tasks, here is exactly what happens step-by-step:

Step A: Dependency Checking (The DAG System)

In software, some tasks cannot start until others finish. For example: *You cannot upload a video (Job B) until the computer finishes rendering the video (Job A).* We built **Directed Acyclic Graph (DAG)** logic. When the jobs arrive, the system immediately locks Job B. Even if Job B is high priority, the AI is not allowed to touch it until Job A officially reports as "COMPLETED" in the Redis database.

Step B: The AI Observation

The AI Brain acts several times a second. It extracts the top 15 available jobs from Redis. It converts their details (Priority, Duration, Time Left) into a matrix of numbers—a format the neural network understands. It passes these numbers through its hidden layers, essentially running probability calculations on which job execution order will yield the highest "Reward".

Step C: Execution

The AI picks the absolute best job to run right now and moves it into the "Deploy" queue. An idle Worker grabs the job, changes its status to `RUNNING`, and executes it.

Step D: The Zombie Killer (Self-Healing)

What happens if the Worker's computer crashes while trying to process the job? We built a self-healing loop. The Scheduler constantly checks the start times of all `RUNNING` jobs. If a job was only supposed to take 5 seconds, but it has been `RUNNING` for 2 minutes, the Orchestrator realizes the worker died. It catches this "Zombie", marks the job as `FAILED`, and safely clears it out so the system doesn't freeze.

---

5. How We Trained the AI

The AI wasn't smart on day one. We used **Reinforcement Learning** (specifically a Deep Q-Network in PyTorch). Reinforcement learning is exactly like training a dog with treats:

- If the AI manages to finish a job before its deadline, we feed the software a mathematical **+1 Reward**.
- If it lets a job sit in the queue so long that the deadline expires, we hit it with a **-1 Penalty**.
- We ran the simulation 500 times at super-speed (`train_model.py`). By the end, the neural network physically rewired its internal weights to understand the perfect balance of prioritizing heavy tasks while preventing starvation for smaller tasks.

---

6. Proof of Success (The Benchmark Results)

To prove our project works, we generated 100 heavily constrained, highly stressful jobs and forced three different algorithms to try and complete them.

- **FIFO Scheduler:** Missed 34 deadlines.
- **Strict Priority:** Missed 38 deadlines (It did the high-priority ones well, but completely starved and killed the low-priority ones).
- **Our AI (DQN):** Missed only **14 deadlines**.

This massive reduction in missed deadlines proves our core theory: **The AI successfully balances importance with urgency, breaking the starvation problem.** By preventing low-priority tasks from waiting indefinitely while still servicing high-priority tasks in time, the Deep Q-Network achieves a significantly higher success rate than static models.

---

7. Real World Applications

While our project was an academic prototype, its architecture is built for the real world. By swapping our "mock" simulated tasks with real python scripts, this exact orchestrator could be used to:

1. **Manage Smart Data Backups:** Safely scheduling large file zip operations on local computers when CPU usage is low.
2. **E-Commerce Order Fulfillment:** Processing high-priority Next-Day-Delivery orders, while intelligently scheduling Standard Shipping updates in the background without slowing down the website.
3. **Cloud Vide/Image Processing:** Queuing thousands of user image uploads and efficiently distributing them to background workers.

---

8. Conclusion

We successfully designed, built, trained, and deployed a self-optimizing orchestration engine. We moved beyond simple code and created a fully decoupled, self-healing Docker architecture backed by Redis databases and real-time Streamlit dashboards. The project successfully met all academic requirements and mathematically proved that Deep Reinforcement Learning can outperform rigid legacy systems in complex scheduling environments.